Assignment 4

CMPT 276: Introduction to Software Engineering
Simon Fraser University

Group 1

Daniil Bugakov 301540700

March 31, 2026

Work Distribution

I reviewed code written by Dev and identified five code smells. My code was reviewed by Brian, who identified four code smells.

Code Smells I Assigned to Dev

The `canMoveTo()` method in `MovingEntity.java` uses an unnecessary if/else structure, reducing readability.

There is code duplication in the `Board` class when converting cell coordinates to pixel coordinates. This occurs in the methods `isWall()`, `isOccupied()`, and `isSpawnable()`.

There is also duplication in cell scanning logic. The same methods repeatedly iterate through `HashSet<Cell>` to check if a cell exists at a given position. A helper method was suggested.

Weak encapsulation is present in `MovingEntity`, where fields `id` and `position` are declared as `protected`, allowing direct modification by subclasses.

A long method exists in the `Guard` class. The `chooseStepToward()` method combines multiple responsibilities, making it difficult to read, maintain, and test.

Code Smells Assigned to Me by Brian

Code-and-paste programming was found in `BankHeist`, where several components used the same hardcoded size values. These were replaced with constants, and a `fillBounds()` helper method was introduced.

Primitive obsession was identified in `BankHeist`, where a primitive value was used in `setDefau ltCloseOperation()` instead of a predefined constant.

A long parameter list was found in the `Cell` constructor. The parameters `x`, `y`, `width`, and `height` were replaced with `Position` and `Dimension` objects. Related classes and tests were updated accordingly.

An anti-pattern was found in `Thief`, where it checked if the player was inside the board, duplicating logic already present in `Board`. Redundant checks were removed, and related tests were adjusted.

Conclusion

The code review helped identify design and maintenance issues, including code duplication, unnecessary complexity, and weak encapsulation. Refactoring improved readability and maintainability. All changes were tested to ensure the game's functionality remained correct.